EH/Unwind Analyzer 1.0.0

Техническое описание для инвесторов и IT-экспертов

Поле	Значение
Product version	1.0.0
Report schema	1.1.0
Windows artifact	artifacts/windows-x64/eh_unwind_analyzer.exe
SHA-256	b1cee8678cdf21d96e2921625ffabc3d5c1be4d41cb022d4d0954ef3692df63a
Final validation	19 июня 2026 года, Windows x64

Документ подготовлен как investor-facing technical package. Он дополняет исходные теоретические материалы в docs/ru и не заменяет их.

Оглавление

1. Краткое резюме для технического инвестора
2. Какую проблему решает продукт
3. Базовая техническая модель Windows x64 unwind
4. Общая архитектура продукта
5. Static analysis: что именно делает проект
6. Heuristic model: как проект ловит аномалии
7. Dynamic mode: зачем он нужен и как работает
8. Report contract и техническая интеграция
9. Проверка качества версии 1.0.0
10. Что можно честно утверждать о правильности anomaly detection
11. Самодостаточность проекта и что стоит добавить дальше
12. Коммерческая логика и инвестиционный тезис
13. Материалы, которые можно дать инвесторам

1. Краткое резюме для технического инвестора

EH/Unwind Analyzer 1.0.0 - defensive tool для анализа Windows PE32+ AMD64 exception/unwind metadata. Проект читает структуру PE-файла, извлекает exception directory, разбирает таблицу RUNTIME_FUNCTION, декодирует UNWIND_INFO, строит набор проверяемых фактов, применяет эвристики аномалий и выпускает отчеты в JSON, HTML, CSV и Ghidra CSV. На Windows x64 дополнительно доступен dynamic mode: target запускается под Windows Debug API, а инструмент собирает runtime evidence по исключениям, handler hits, VEH/VCH/UEF и runtime function tables.

Для инвестора важен не только сам анализатор, но и состояние зрелости продукта. Версия 1.0.0 имеет зафиксированный Windows artifact, schema contract 1.1.0, проверенный SHA-256, два независимых Windows toolchain-прохода, статическую проверку на synthetic/labeled и real benign corpus, динамическую проверку на smoke-сценариях и full dynamic corpus, а также release hygiene gates. Это означает, что проект можно презентовать как работающий technical core, а не как исследовательский прототип без воспроизводимой сборки.

Важная позиция проекта: инструмент не заявляет себя как универсальный антивирус и не выдает heuristic score за математическую вероятность вредоносности. Он дает explainable evidence по конкретному низкоуровневому механизму Windows. Это делает его особенно полезным для malware labs, reverse engineering teams, sandbox vendors, EDR research, supply-chain scanning Windows binaries и технического due diligence.

- Релиз: 1.0.0, report schema: 1.1.0.
- Финальная локальная валидация: 19 июня 2026 года, Windows x64.
- Artifact: artifacts/windows-x64/eh_unwind_analyzer.exe.
- SHA-256: b1cee8678cdf21d96e2921625ffabc3d5c1be4d41cb022d4d0954ef3692df63a.
- Static analysis cross-platform; dynamic mode - Windows x64 only.

2. Какую проблему решает продукт

Большинство security tools хорошо смотрит на очевидные слои: imports, strings, sections, behavioral telemetry, network activity, process tree. Но Windows x64 exception/unwind layer часто остается зоной для ручного анализа. В этом слое находятся данные, которые Windows использует для восстановления стека, перехода по exception handlers, C++ exception handling и корректной работы отладчиков.

Если attacker, protector или packer ломает, сжимает, перенаправляет или динамически регистрирует unwind metadata, обычный analyst workflow получает дополнительную неопределенность. Stack trace становится менее надежным, статический control flow труднее интерпретировать, exception path может уходить в handlers, которые не выглядят как обычные ветки кода, а runtime-регистрация function tables может существовать только во время выполнения.

EH/Unwind Analyzer превращает этот слой в отдельный продуктовый сигнал. Он не заменяет sandbox, EDR или disassembler. Он дополняет их: берет узкий, но технически важный слой Windows ABI и делает по нему повторяемый, машинно-валидируемый и удобный для аналитика отчет. Для рынка это означает возможность добавить low-level explainable signal туда, где сейчас часто есть только manual expert work.

Кто получает пользу	Практический сценарий	Как используется результат
Malware lab / reverse engineering	быстрый triage packed/protected PE	HTML/JSON report, Ghidra CSV, manual validation steps
Sandbox vendor	добавить Windows ABI signal к behavioral telemetry	batch CLI runner, schema validation, derived risk fields
EDR research	исследовать exception-flow evasion и handler behavior	dynamic evidence timeline and correlation summary
Supply-chain/AppSec	проверять Windows binaries до поставки	static mode in CI, policy thresholds, archived JSON evidence

3. Базовая техническая модель Windows x64 unwind

В PE32+ AMD64 exception directory обычно указывает на .pdata. В .pdata находится массив записей RUNTIME_FUNCTION. Каждая запись имеет BeginAddress, EndAddress и UnwindInfoAddress. Диапазон функции трактуется как half-open interval: [BeginAddress, EndAddress). Третье поле указывает на UNWIND_INFO, который обычно лежит в .xdata или связанном read-only metadata section.

UNWIND_INFO описывает, как восстановить stack frame: какие регистры сохранены, какой размер stack allocation, есть ли frame register, есть ли exception handler, termination handler или chained unwind record. Tail-часть UNWIND_INFO зависит от flags и выравнивания CountOfCodes. Ошибка в этих деталях не является косметикой: она меняет то, как Windows будет искать handler или восстанавливать контекст.

Проект поддерживает UNWIND_INFO V1/V2, включая V2 epilog descriptors. Preview V3 намеренно reported as unsupported. Это правильная инженерная позиция: лучше явно сказать, что формат не поддержан, чем неправильно декодировать новый layout и дать инвестору или аналитику ложную уверенность.

Объект	Роль	Что проверяет проект
PE headers	параметры образа, sections, SizeOfImage	границы, alignment, overlaps, RVA mapping
Exception directory	адрес и размер таблицы .pdata	OOB, размер кратный 12, placement
RUNTIME_FUNCTION	диапазон функции и ссылка на UNWIND_INFO	Begin/End ordering, OOB RVAs, overlaps, sorting
UNWIND_INFO	stack unwind instructions and handlers	version, flags, codes, tail layout, chain depth
Runtime function table	динамически зарегистрированные диапазоны	RtlAddFunctionTable and callback registration evidence

4. Общая архитектура продукта

Core проекта - модульный C17 CLI. Это не распределенная система и не сложный backend; для текущей стадии это плюс. Один проверенный executable легче поставлять, запускать в CI, запускать в sandbox worker и оборачивать в API. Внутренние границы модулей при этом достаточно явные: PE parsing, unwind decoding, heuristic assessment, dynamic collection, report writing and release metadata.

CLI принимает target path, параметры анализа и output options. Static mode доступен на разных платформах, потому что он только читает Windows PE file. Dynamic mode компилируется и работает на Windows x64, потому что использует Windows Debug API и должен запускать target process. Outputs могут создаваться в одном запуске: console, JSON, HTML, CSV, Ghidra CSV.

Такая архитектура хорошо подходит для коммерциализации. Можно оставить core как стабильный analyzer binary, а вокруг него строить batch service, dashboard, API, подписанные release packages, policy packs и интеграции в sandbox/EDR/SOAR. Это снижает риск: product layer можно развивать без переписывания parser и эвристик.

Модуль	Ответственность	Почему важно для инвестора
main.c	CLI, options, orchestration	stable entry point for automation
pe.c	PE validation and RVA mapping	base correctness and memory-safety boundary
unwind.c	RUNTIME_FUNCTION and UNWIND_INFO decoding	technical differentiation
heuristics.c	findings, score, confidence, notes	explainable assessment model
dynamic_win.c	Windows Debug API collector	runtime corroboration layer
report.c	JSON/HTML/CSV/Ghidra outputs	integration and auditability

5. Static analysis: что именно делает проект

Static analysis начинается с безопасного чтения файла и проверки PE layout. Инструмент проверяет headers, section ranges, raw/virtual overlaps, alignments, SizeOfImage, exception directory placement and boundaries. Это важно, потому что ошибочная RVA mapping способна испортить все последующие выводы.

Затем analyzer читает все RUNTIME_FUNCTION records и декодирует связанные UNWIND_INFO. Для каждой функции сохраняется begin RVA, end RVA, unwind info RVA, размер, section, parse status, operations summary, handler/chained metadata and exact parse failure reason. Если decode не удался, причина не теряется: она идет в отчет как evidence, а не как безымянная ошибка.

Static mode не исполняет sample. Поэтому его можно безопасно запускать на больших наборах файлов, в CI, в supply-chain проверке, в pre-triage или в offline lab. Dynamic mode нужен только тогда, когда нужно подтвердить runtime behavior.

- Проверяются границы PE, sections, exception directory and RVA mapping.
- RUNTIME_FUNCTION читается как массив 12-byte records.
- Диапазоны функций трактуются как [BeginAddress, EndAddress).
- UNWIND_INFO декодируется с учетом V1/V2 layout, flags, operations, tail alignment and chain rules.
- Каждый parse failure сохраняет точную причину, пригодную для analyst review.

6. Heuristic model: как проект ловит аномалии

Эвристики проекта построены как explainable rule system. Это не ML model, не signature database и не вероятностный malware verdict. Каждое срабатывание имеет code, severity, family, affected count, representative RVA/section and explanation. Итоговый score 0..100 помогает ранжировать samples для triage, но не является доказательством вредоносности.

Evidence families позволяют понять, сигнал одиночный или подтвержден несколькими слоями. Например, один unusual placement может быть weak evidence, но unusual placement вместе с corrupted UNWIND_INFO, handler dominance и runtime handler hit становится существенно сильнее. Confidence и weakening rules нужны для предотвращения агрессивных false positives.

Важный результат проверки 1.0.0: real benign corpus из 106 файлов дал 106 валидных отчетов, 174,795 parsed runtime functions и ноль non-zero heuristic scores. Это не доказывает отсутствие всех возможных false positives в мире, но показывает, что текущие правила не шумят на проверенном benign наборе и пригодны для презентации как аккуратный triage signal.

Семейство сигналов	Примеры	Интерпретация
RUNTIME_FUNCTION integrity	Begin >= End, OOB RVA, bad table size	структурное нарушение .pdata
UNWIND_INFO integrity	bad version, invalid tail, chain problem	нарушение decoder/ABI expectations
Metadata placement	writable/executable section, unusual section	потенциальная изменяемость или маскировка metadata
Reuse/statistical patterns	shared UnwindInfoRVA, dominance	protector-like or compacted layout
Runtime corroboration	handler hit, RtlAddFunctionTable, VEH/VCH/UEF	подтверждение активности механизма во время выполнения

7. Dynamic mode: зачем он нужен и как работает

Static analysis отвечает на вопрос: что записано в файле. Dynamic mode отвечает на другой вопрос: какие exception-related механизмы реально активны во время выполнения. На Windows x64 analyzer запускает target под debugger, отслеживает debug events, ставит breakpoints на handlers referenced by static UNWIND_INFO, а также на runtime function table APIs and exception handler registration paths.

Dynamic report фиксирует first- and second-chance exceptions, hits of unwind handlers, VEH/VCH/UEF registration and handler hits, calls to RtlAddFunctionTable and RtlInstallFunctionTableCallback, bounded RIP trace after handler hit, target exit status, timeout state and structured event timeline. Также есть instrumentation-completeness counters: modules, breakpoints, re-arming, dropped events.

Важно: dynamic mode исполняет target. Для untrusted samples он должен запускаться в isolated VM/sandbox. Это не недостаток, а нормальная граница безопасности. Проект документирует ее и не пытается скрывать execution risk.

Observed item	Что фиксируется	Зачем это нужно
Exceptions	first/second chance, code, address, stage	понимать exception path
Static handler hits	breakpoint on handlers from UNWIND_INFO	связать static metadata с runtime

Observed item	Что фиксируется	Зачем это нужно
VEH/VCH/UEF	registration and handler activity	видеть нестандартную обработку исключений
Runtime tables	RtlAddFunctionTable and callback APIs	выявить metadata, созданную после загрузки
Completeness counters	breakpoints, modules, dropped events, timeout	оценить качество dynamic session

8. Report contract и техническая интеграция

Проект производит несколько типов output. JSON - основной machine-readable contract: он содержит metadata, tool version, schema version, input hash, PE validation, section data, functions, heuristic_analysis, dynamic_validation and final_assessment. HTML - self-contained analyst dashboard без сетевых assets. CSV дает плоскую таблицу функций. Ghidra CSV предназначен для importer script.

Для инвестора это важно потому, что продукт уже имеет integration surface. Backend может запускать analyzer как subprocess, валидировать report.schema.json, сохранять raw report, индексировать selected fields и строить dashboard. Для reverse engineering workflow Ghidra importer открывает путь к comments/bookmarks без ручного копирования.

Report schema versioning снижает риск будущих изменений. Новые поля можно добавлять через schema decision and tests. Это делает проект пригодным не только для demo, но и для пилотов, где отчеты должны быть стабильными, сравнимыми и архивируемыми.

- JSON: authoritative contract, schema 1.1.0.
- HTML: self-contained dashboard для аналитика.
- CSV: плоская выгрузка function/unwind facts.
- Ghidra CSV: импорт range/comments/bookmarks в reverse engineering workflow.
- Reports include build provenance, input SHA-256 and final assessment fields.

9. Проверка качества версии 1.0.0

Финальная проверка версии 1.0.0 проведена на Windows x64 19 июня 2026 года. Проект проверялся на двух независимых Windows toolchains: MinGW-w64 GCC 13.2.0 и MSVC 19.44.35215. Это снижает риск компилятор-специфичных проблем и показывает, что C17 codebase не завязан на один toolchain.

Static validation разделена на synthetic/labeled corpus и real benign corpus. Labeled corpus нужен для проверки, что известные аномалии действительно поднимаются эвристиками. Benign corpus нужен для калибровки шума: на проверенном наборе реальных benign PE-файлов score остался 0.

Dynamic validation включает 4 targeted smoke-сценария и full dynamic corpus из 124 inputs. Full corpus дал 124/124 schema/semantic-valid reports. Из 86 started debug sessions 54 завершились completed, 32 были controlled timeouts, 8 targets потребовали elevation, 30 DLL/PYD были корректно классифицированы как

unsupported_library_launch. Unexpected failures: 0. Dropped events: 0.

Gate	Результат
MinGW-w64 Release CTest	16/16 PASS
MSVC Release CTest	16/16 PASS
Static labeled corpus	14/14 valid; 2 benign score 0; 12 anomalous flagged
Real benign corpus	106/106 valid; 174,795 runtime functions parsed; zero non-zero scores
Dynamic smoke	4/4 PASS: VEH, VCH, UEF, RtlFunctionTable
Full dynamic corpus	124/124 valid reports; unexpected failures 0; dropped events 0
Release artifact hash gate	PASS, SHA-256 recorded in release manifest

10. Что можно честно утверждать о правильности anomaly detection

По текущей evidence базе можно утверждать, что parser and report pipeline работают устойчиво на проверенных наборах, что synthetic anomalous samples поднимаются эвристиками, что benign corpus не создает non-zero scores, и что dynamic collector формирует валидные отчеты с controlled outcomes. Это достаточная база для investor presentation and pilot discussion.

Нельзя утверждать, что инструмент обнаруживает все возможные виды обфускации, все malware families или все будущие версии Windows unwind metadata. Он не сравнивает пока UNWIND_INFO operations с дизассемблированным machine prolog, не поддерживает PE32/x86/ARM64 и не декодирует V3 preview layout. Эти пункты должны быть roadmap, а не скрытыми обещаниями.

Правильная инвестиционная формулировка: проект уже надежно закрывает выбранный narrow technical domain и имеет проверенный путь к расширению. Узость domain не слабость, а коммерческая специализация: explainable low-level signal легче продать как дополнение к sandbox/EDR/RE workflow, чем как очередной общий malware classifier.

- Подтверждено: корректность build/test/report/release gates для 1.0.0.
- Подтверждено: отсутствие шума на проверенном real benign corpus.
- Подтверждено: synthetic/labeled anomalies детектируются текущей policy.
- Подтверждено: dynamic collector дает валидные reports и controlled outcomes.
- Не заявляется: универсальное обнаружение malware или статистическая probability model.

11. Самодостаточность проекта и что стоит добавить дальше

Для презентации инвесторам проект самодостаточен: есть executable, source, docs, Russian theory/defense materials, release status, product readiness document, tests, validation evidence and clean release bundles. Пользователь может показать не только slides, но и реальные команды, generated reports, schema and

validation matrix.

Для коммерческого пилота стоит добавить signed Windows build, короткий GUI/workbench, API wrapper, sample report gallery, installer packaging, DLL host runner for library samples, larger curated corpus, policy configuration and clearer licensing/commercial packaging. Эти additions не блокируют презентацию 1.0.0, но делают следующий этап убедительнее для enterprise buyers.

На технической стороне главные расширения: disassembly correlation, UNWIND_INFO V3 support after formal spec/test coverage, richer typed dynamic event schema, remote memory decoding for dynamic function tables, per-policy thresholds and team dashboard. Это естественный roadmap после инвестиций.

Состояние сейчас	Достаточно для презентации?	Что даст следующий этап
CLI analyzer + reports	да	batch/API/GUI для пилотов
Validation evidence 1.0.0	да	больший corpus и external validation
Ghidra CSV importer	да	глубже интеграция в RE workflow
Unsigned local Windows artifact	для demo да	signed build для enterprise доверия
Dynamic EXE launch	да	DLL host runner расширит coverage

12. Коммерческая логика и инвестиционный тезис

Продукт находится в удобной для инвестора точке: core уже работает и проверен, но коммерческая упаковка еще может дать сильный рост ценности. Это снижает early technical risk и оставляет понятные направления использования средств: product UX, integrations, corpus, signing, pilot support and sales engineering.

Позиционирование лучше строить не как "замена антивируса", а как specialized engine для анализа Windows exception/unwind anomalies. Такой engine можно лицензировать лабораториям, sandbox vendors, EDR research teams, reverse engineering groups and AppSec teams. Его сила - объяснимость: каждое срабатывание можно показать через RVA, section, record, handler, report field and validation step.

Инвестиционная гипотеза: рост packed/protected binaries, AI-assisted malware development и усложнение analyst workload повышают спрос на узкие, проверяемые, интегрируемые сигналы. EH/Unwind Analyzer закрывает именно такой signal layer.

Горизонт	Технический результат	Бизнес-результат
0-90 дней	signed builds, GUI prototype, API wrapper, report gallery	pilot-ready package
90-180 дней	DLL host, disassembly correlation, policy versioning	higher precision and wider coverage
180-365 дней	sandbox/EDR connectors, dashboard, policy packs	enterprise proof of value

13. Материалы, которые можно дать инвесторам

В папке docs/ru/преза подготовлены три файла. PPTX нужен для живого выступления и технического разговора со специалистами инвестора. PDF нужен как самостоятельный документ, который можно отправить заранее или после встречи. Markdown нужен как редактируемая текстовая версия для быстрого обновления, вставки в data room или подготовки англоязычной версии.

Исходные 5 файлов в docs/ru остаются отдельной теоретической и защитной базой проекта. Они не являются временными файлами и должны сохраняться. Новая папка docs/ru/преза не заменяет их, а добавляет investor-facing technical package.

- EH_Unwind_Analyzer_Техническая_презентация_1.0.0_RU.pptx - 15-slide technical deck.
- EH_Unwind_Analyzer_Техническое_описание_1.0.0_RU.pdf - полный PDF-документ для чтения.
- EH_Unwind_Analyzer_Техническое_описание_1.0.0_RU.md - редактируемый источник описания.
- ZIP в финальной папке не нужен и не включается.

Финальный вывод

EH/Unwind Analyzer 1.0.0 можно показывать инвесторам как verified technical core: есть работающий Windows artifact, проверенный parser/report pipeline, explainable heuristics, Windows dynamic evidence collector, schema-based reports, Ghidra integration and documented limitations. Следующий инвестиционный этап должен быть направлен на productization: signed builds, API/GUI, расширенный corpus, integrations and pilot packaging.